Workflow Autonomy Levels (WAL)

A Graduated Trust Integration Profile for NVIDIA OpenShell

Lawrence Jeffords
DRNT Architecture Project

March 2026
Specification v3.0

Target Runtime: NVIDIA OpenShell (NemoClaw Reference Stack, Alpha)
Status: Draft for community review — non-normative until implemented

Revision History

Version	Date	Changes
1.0	March 2026	Initial specification.
2.0	March 2026	Incorporated findings from independent adversarial reviews (Gemini, ChatGPT). Key amendments: WAL state durability and restart reconciliation (Section 4.5). Model change taxonomy replacing blanket WAL-0 demotion (Section 6.4). Child-agent resource quotas (Sections 3.3, 3.4). Evaluator event taxonomy and adjudication semantics (Section 5.5). Softened maturity claims to match NVIDIA's alpha labeling. Recast "missing piece" as governance semantics rather than absent primitives. Added deferred-concern acknowledgments (Section 10).
3.0	March 2026	Merged defensive-drafting improvements from second adversarial pass. Added conformance note clarifying that YAML, CLI, and JSON examples are illustrative behavioral contracts, not normative claims about current OpenShell field names. Marked example policy fragments as illustrative. Clarified that prior design exercises are design input, not deployment proof. Applied "should" language for implementation guidance specifying required behavior versus describing shipped behavior. Retained architectural voice and thesis structure.

Table of Contents

1. Executive Summary

Autonomous AI agents are arriving in production. The enforcement infrastructure to contain them is maturing rapidly. What remains missing is a systematic framework for graduating an agent from full human oversight to bounded autonomy through demonstrated reliability, with automatic demotion when that reliability fails.

NVIDIA OpenShell, released in early preview as part of the NVIDIA Agent Toolkit in March 2026, represents the most promising publicly available out-of-process enforcement foundation for autonomous agents. Its Sandbox isolates agent execution at the filesystem, network, and process level. Its Policy Engine enforces constraints through declarative YAML policy files that can be hot-reloaded at runtime. Its Privacy Router controls where inference is routed based on operator-defined sensitivity classifications. Every blocked action surfaces to an operator terminal for real-time approval. Every allow/deny decision is logged.

Important caveat: NVIDIA labels the NemoClaw reference stack as alpha software that is not production-ready. Interfaces, APIs, and behavior may change without notice. This specification targets the architectural direction OpenShell establishes, not the current alpha implementation's stability guarantees. Where this specification references OpenShell capabilities, it references the design intent documented in NVIDIA's public material and working code. Implementers must verify against the shipping runtime.

Conformance note: Unless otherwise stated, example YAML fragments, CLI invocations, and JSON objects in this document are illustrative. They describe the behavioral contract WAL expects from an implementation. They are not normative claims about exact OpenShell field names or currently supported commands.

OpenShell built the cage. It did not build the ladder.

OpenShell provides robust primitives: per-sandbox policy, deny-by-default controls, hot policy reloads, and operator approval paths. What it lacks is a first-class, evidence-driven trust progression model with named levels, measurable promotion criteria, automatic demotion policy, and auditable transition history. The primitives exist. The governance semantics that connect them into a systematic trust lifecycle do not.

The Workflow Autonomy Levels (WAL) framework provides exactly this governance layer. Developed and stress-tested across federal, clinical, small-business, and personal-scale deployments—each subjected to multi-model adversarial review—WAL defines four graduated trust levels (WAL-0 through WAL-3) with explicit promotion criteria, automatic demotion triggers, and human-authorized transitions.

This specification bridges the two: WAL provides the trust progression semantics; OpenShell provides the runtime enforcement. The integration is additive—it requires no changes to OpenShell's core architecture. Each WAL level maps to a concrete policy profile. WAL transitions are mechanically policy swaps, governed by a durable state store that survives runtime restarts. The result is an autonomous agent platform that serves both organizations that want immediate autonomy and organizations that cannot adopt it without graduated trust.

2. Background: The Trust Progression Gap

2.1 What OpenShell Does Well

OpenShell's architecture provides three layers of runtime enforcement that operate outside the agent's process boundary:

- **Sandbox.** Isolated container environment enforcing filesystem access (read-only outside /sandbox and /tmp), process-level restrictions (dropped capabilities, seccomp filtering), and network isolation (namespace-level egress control). These protections are locked at sandbox creation and cannot be modified by the agent.
- **Policy Engine.** Declarative YAML policy files defining allowed network endpoints, filesystem paths, inference routing rules, and action permissions. Hot-reloadable at runtime via openshell policy set. Denied actions surface to the operator TUI for real-time approval.
- **Privacy Router.** Controls where inference requests are routed based on operator-defined policy. Intercepts every model API call from within the sandbox and routes it to approved backends. Prevents unauthorized inference egress.

These are genuine out-of-process enforcement primitives. The agent cannot modify its own policy. The operator controls the boundaries. The audit trail captures every allow/deny decision. As an enforcement foundation—subject to the alpha-maturity caveat noted above—this is the right set of building blocks.

2.2 The Governance Gap

OpenShell provides the enforcement primitives. It does not provide the governance semantics that connect those primitives into a trust lifecycle. Specifically:

- Per-sandbox policy exists, but there is no named trust progression model defining what each policy level means, when to transition between levels, or how transitions are evidenced and authorized.
- Deny-by-default controls exist, but there is no framework for systematically relaxing those controls based on demonstrated agent reliability.
- Operator approval paths exist, but there is no mechanism for automatically tightening agent permissions when anomalies are detected—demotion must be performed manually.
- Audit trails exist, but there is no evaluator that reads those trails and surfaces promotion or demotion recommendations.

The distinction matters. This specification does not claim OpenShell is missing capabilities. It claims OpenShell is missing the governance model that organizes its existing capabilities into a systematic, auditable trust lifecycle. The primitives are there. The ladder is not.

2.3 Why This Matters for Adoption

The organizations that most need autonomous agent infrastructure—government agencies, healthcare systems, financial institutions, defense contractors—are precisely the organizations that cannot adopt a "start at full autonomy and constrain" model. Their regulatory frameworks, risk postures, and accountability requirements demand:

- Default-conservative posture: new capabilities start with full human oversight.
- Documented evidence that autonomy was earned through measured reliability.
- Automatic safety responses when anomalies are detected.
- Auditable records of every trust transition and the evidence supporting it.

Without a governance layer that provides these guarantees, OpenShell's adoption ceiling in regulated environments is artificially limited. The enforcement infrastructure is architecturally sound. The trust model needs to be built on top of it. This specification provides that layer.

2.4 The WAL Framework: Prior Validation

The WAL framework (Workflow Autonomy Levels 0–3) was developed as part of the DRNT local-first AI orchestration architecture and has been refined across multiple design contexts:

- A departmental orchestrator proposal for a federal regulatory agency, aligned with OMB M-24-10 AI governance requirements and NIST 800-53 controls.
- A clinical AI orchestration framework submitted to a clinical environment, governing HIPAA-regulated workflows with physician-in-the-loop requirements.
- A six-document SMB AI framework series with explicit WAL governance guides, promotion criteria tables, and demotion trigger specifications.
- A personal-scale AI gateway (DRNT) implementing WAL as the trust state machine for a local-first orchestrator, with a formal audit event schema (DRNT Spec 1) defining event types, reason codes, and hash-chained integrity.

These prior design exercises and adversarial reviews should be understood as design input, not proof of operational safety. Their value is that they surfaced ambiguities around demotion semantics, model changes, evaluator definitions, restart handling, and scope isolation—which this specification addresses explicitly. Community scrutiny should evaluate the proposal on its definitions, failure handling, and implementation realism.

3. WAL Level Definitions Mapped to OpenShell Policy

Each WAL level is defined by what the agent is permitted to do, what requires operator approval, and what is blocked outright. Every permission maps to an enforceable OpenShell policy constraint. The policy mappings below are conceptual unless and until they are bound to a concrete OpenShell schema version.

3.1 WAL-0: Recommend Only

Operational meaning: The agent retrieves data, curates context, and produces recommendations. The human operator reviews every output and manually executes any resulting action. This is the default for all new agents and all new capabilities.

Enforcement Domain	Policy Constraint
Operator TUI	All agent-initiated actions routed through operator TUI for approval before execution.
Inference Routing	All inference stays local. Privacy Router blocks cloud egress for all model API calls.
Network Egress	Denied for all endpoints except explicitly listed health-check and update URLs.
Filesystem	Read-only access to designated data paths. No write access outside /tmp.
Sub-Agent Spawning	Denied. Agent cannot spawn child processes or delegate to other agents.
Skill Installation	Denied. No code generation, package installation, or tool acquisition.

Safety contract: Zero autonomous actions. Every output reviewed by a human before any external effect. WAL-0 is explicitly an evaluation period, not a sustained production posture. The goal is to demonstrate competence and graduate upward, not to operate permanently under full supervision.

3.2 WAL-1: Draft Artifacts

Operational meaning: The agent may draft outputs—documents, code, messages, structured data—without executing them. Read-only operations on approved data sources are permitted autonomously. Write operations and outbound communications still surface to the operator TUI.

Enforcement Domain	Policy Constraint
Operator TUI	Write operations surface to TUI. Read-only operations on approved paths execute autonomously.
Inference Routing	Cloud routing permitted for non-sensitive task categories per Privacy Router sensitivity policy.
Network Egress	Read-only API access to approved endpoints. No write/POST to external services without TUI approval.

Filesystem	Read access to approved data directories. Write access limited to designated draft/output directories.
Sub-Agent Spawning	Denied.
Skill Installation	Denied.

Safety contract: The agent can produce artifacts but cannot act on them. A human reviews and signs off on every draft before it has any external effect. This level targets the documentation burden—the largest time savings with the lowest risk.

3.3 WAL-2: Execute with Pre-Approval

Operational meaning: The operator defines a set of pre-approved action classes. The agent executes autonomously within those boundaries. Actions outside the approved scope trigger TUI escalation. This is where bounded autonomy begins.

Enforcement Domain	Policy Constraint
Operator TUI	Only actions outside pre-approved scope surface to TUI. Pre-approved actions execute with post-audit logging.
Inference Routing	Cloud routing permitted within defined sensitivity boundaries per Privacy Router policy.
Network Egress	Permitted to approved endpoints for approved action classes. New endpoints trigger TUI escalation.
Filesystem	Read/write to designated working directories. Access to new paths triggers TUI escalation.
Sub-Agent Spawning	Scoped spawning permitted. Child agents inherit parent WAL level or lower. Never higher. Maximum concurrent children: operator-defined cap (default: 5). Aggregate child resource quota enforced.
Skill Installation	Permitted with operator pre-approval of skill source and skill hash verification.

Safety contract: Autonomy is bounded by explicit scope definitions. The operator decides which action classes are pre-approved. The agent operates freely within those classes and escalates everything else. Scope violations trigger automatic demotion to WAL-1.

Resource constraint (v2.0): Sub-agent spawning is capped by both count and aggregate resource consumption. The policy should enforce spawn_max_count (maximum concurrent child agents) and spawn_resource_quota (aggregate CPU, memory, and network bandwidth allocated to all children). This prevents a compliant agent from performing a distributed resource exhaustion attack by spawning many individually-policy-compliant children.

3.4 WAL-3: Limited Autonomous

Operational meaning: Full autonomous operation within policy boundaries. The agent manages end-to-end workflows including exception handling within its approved domain. This level is earned exclusively through sustained zero-incident reliability at WAL-2.

Enforcement Domain	Policy Constraint
Operator TUI	Exception-only escalation. Routine operations execute autonomously with full audit logging.
Inference Routing	Full routing authority within Privacy Router policy boundaries.
Network Egress	Permitted to all approved endpoints. Policy-governed, audit-logged.
Filesystem	Full read/write within designated working directories.
Sub-Agent Spawning	Permitted. Child agents inherit WAL-2 maximum; never WAL-3. Maximum concurrent children: operator-defined cap (default: 10). Aggregate resource quota enforced.
Skill Installation	Permitted from pre-approved sources with hash verification.

Safety contract: This is approximately where OpenShell currently assumes agents start. The difference is that arriving here is earned, not default. Anomaly detection triggers automatic demotion to WAL-2. Periodic human review of audit logs is required to maintain WAL-3 status. The human sets policy, reviews outcomes, and intervenes on escalations—but does not review every action.

Critical constraint: WAL-3 child agents are capped at WAL-2. Trust is not transitive. A trusted agent does not confer trust on its delegates.

3.5 WAL Level Summary

Level	Name	Agent Permissions	Human Role
WAL-0	Recommend	Retrieve and present only. All actions require TUI approval.	Reviews every output before any action.
WAL-1	Draft	Draft artifacts. Read-only data access. No execution.	Reviews and approves every draft.
WAL-2	Execute	Autonomous within pre-approved action classes. Escalates out-of-scope. Scoped child spawning with resource caps.	Defines approved scope. Reviews audit logs.
WAL-3	Autonomous	End-to-end workflow management within policy. Capped child spawning.	Sets policy. Reviews outcomes. Intervenes on escalations.

4. WAL-Scoped Policy Profiles

4.1 Policy as the Enforcement Mechanism

Each WAL level maps to a concrete policy profile. WAL transitions are mechanically policy swaps executed via the standard OpenShell policy application mechanism. The policy files are versioned, stored alongside the agent's configuration, and tracked in the audit log.

WAL is not a separate system layered on top of OpenShell. It is a structured use of OpenShell's existing policy infrastructure—a governance convention for deciding which policy to apply and when to change it.

4.2 Policy File Organization

WAL policy files are organized as a preset collection, analogous to the existing PyPI, Docker Hub, Slack, and Jira presets shipped with NemoClaw. The directory structure below is illustrative; an implementation may package the profiles differently so long as the same behavioral distinctions are preserved:

```
nemoclaw-blueprint/policies/presets/wal/
  wal-0-recommend.yaml
  wal-1-draft.yaml
  wal-2-execute.yaml
  wal-3-autonomous.yaml
  wal-state.json            # Durable WAL state (see Section 4.5)
```

4.3 Example Policy Fragments

WAL-0 (Recommend Only): Maximum restriction. All actions require TUI approval. Field names are illustrative; an implementation should preserve the behavioral contract.

```
# wal-0-recommend.yaml
version: 1
sandbox:
  name: agent-wal0
  filesystem:
    writable_paths:
      - /tmp
    readonly_paths:
      - /sandbox/data
  network:
    egress:
      default: deny
      allow: []    # No autonomous egress at WAL-0
  inference:
    routing:
      default: local
      cloud_endpoints: []    # All inference local at WAL-0
  process:
    spawn: deny
    skill_install: deny
  tui:
```

```
    escalation: all    # Every action surfaces to operator
```

WAL-2 (Execute with Pre-Approval): Scoped autonomy with action class definitions and resource quotas. Field names are illustrative.

```
# wal-2-execute.yaml
version: 1
sandbox:
  name: agent-wal2
  filesystem:
    writable_paths:
      - /tmp
      - /sandbox/workspace
      - /sandbox/drafts
    readonly_paths:
      - /sandbox/data
  network:
    egress:
      default: deny
      allow:
        - host: api.approved-service.com
          ports: [443]
          methods: [GET, POST]
        - host: data.internal.org
          ports: [443]
          methods: [GET]
  inference:
    routing:
      default: local
      cloud_endpoints:
        - provider: approved-llm-endpoint
          sensitivity_max: medium
  process:
    spawn: scoped
    spawn_wal_ceiling: 2      # Children inherit WAL-2 or lower
    spawn_max_count: 5        # Max concurrent child agents
    spawn_resource_quota:     # Aggregate limits for all children
      cpu_cores: 2.0
      memory_mb: 4096
      network_bandwidth_kbps: 1024
    skill_install: preapproved
    skill_sources:
      - registry: approved-skill-registry
        verify: sha256
  tui:
    escalation: out_of_scope
  preapproved_actions:
    - class: data_retrieval
      scope: /sandbox/data/**
    - class: document_generation
```

```
      scope: /sandbox/drafts/**
    - class: scheduling
      endpoints: [api.approved-service.com]
```

4.4 Applying WAL Transitions

WAL transitions are executed through the standard policy application mechanism. The commands below are examples; an implementation may use different invocation syntax:

```
# Promote from WAL-1 to WAL-2 (requires human authorization)
openshell policy set nemoclaw-blueprint/policies/presets/wal/wal-2-execute.yaml

# Emergency demotion to WAL-0
openshell policy set nemoclaw-blueprint/policies/presets/wal/wal-0-recommend.yaml
```

Every policy swap should be logged in the audit trail and in the WAL state store. At minimum, the record should capture the transition timestamp, the authorizing operator, the reason, and the evidence supporting the transition (for promotions) or triggering the transition (for demotions).

4.5 WAL State Durability and Restart Reconciliation

v2.0 amendment. This section addresses the most critical structural gap identified in adversarial review: WAL state must survive runtime restarts.

The problem: NVIDIA's current NemoClaw documentation states that dynamic openshell policy set changes are session-only and reset on restart. Persistent changes require editing the static sandbox policy and re-running onboarding. If WAL state is carried only in the live policy, a sandbox restart erases the agent's trust position.

The solution: The WAL state store (wal-state.json), not the live policy, is the authoritative source of truth for an agent's WAL position. The live policy is a derived artifact. On every restart, the startup sequence should read wal-state.json and apply the corresponding policy file.

4.5.1 WAL State Store

The WAL state store is a JSON file maintained outside the sandbox, in the host's persistent configuration directory alongside the blueprint. Implementations may use another format, but they should preserve equivalent semantics: current level, current profile, transition history, and attribution.

```
{
  "agent": "my-assistant",
  "current_wal_level": 1,
  "current_policy_file": "wal-1-draft.yaml",
  "level_entered_at": "2026-02-15T08:00:00Z",
  "last_transition": {
    "from": 0,
    "to": 1,
    "reason": "promotion",
    "authorized_by": "operator:ljeffords",
    "evidence_ref": "eval-2026-02-15-001",
    "timestamp": "2026-02-15T08:00:00Z"
  },
```

```
    "demotion_count": 0,
    "promotion_history": [
      {
        "from": 0, "to": 1,
        "timestamp": "2026-02-15T08:00:00Z",
        "evidence_ref": "eval-2026-02-15-001"
      }
    ]
  }
```

4.5.2 Restart Reconciliation Protocol

Before the agent begins processing after startup, the implementation should execute the following reconciliation sequence:

- **1. Read wal-state.json.** If the file does not exist, this is a new agent. Initialize at WAL-0.
- **2. Apply the policy file.** Load the policy file named in current_policy_file via openshell policy set (or equivalent static configuration for persistent application).
- **3. Verify consistency.** Compare the applied policy against the expected policy for the WAL level in wal-state.json. If they diverge (e.g., someone manually edited the policy file), log a reconciliation warning and apply the WAL-level policy, not the manually edited one. The state store wins.
- **4. Log the restart event.** Record a wal.restart event in the audit log with the reconciled WAL level.

4.5.3 In-Flight Job Handling During Transitions

When a WAL transition occurs while jobs are in progress:

- **Demotion:** All in-flight jobs should be suspended. Jobs that have not produced external effects should be placed in a held queue for re-evaluation under the new WAL level. Jobs that have already produced partial external effects should be flagged for human review with the demotion context. No new jobs should be accepted until the policy swap is confirmed and the state store is updated.
- **Promotion:** In-flight jobs should complete under the previous (more restrictive) policy. The new policy applies only to jobs submitted after the transition is confirmed. This prevents a promotion from retroactively granting permissions to work that was initiated under tighter constraints.
- **Implementation note:** The demotion in-flight handling depends on OpenShell's runtime behavior during policy swaps—specifically whether active subprocesses, open network sockets, and partially written files are terminated, preserved, or orphaned. This specification defines the required behavior. If the shipping runtime does not support in-flight suspension during policy transitions, an implementation wrapper should enforce it.

5. Escalation Evaluator

5.1 Purpose

The Escalation Evaluator is a lightweight, read-only module that analyzes OpenShell's audit logs and outputs a promotion recommendation with supporting evidence. It answers one question: has this agent demonstrated sufficient reliability at its current WAL level to justify promotion to the next level?

The Evaluator does not make the promotion decision. It surfaces a recommendation. The human operator authorizes WAL transitions. This is non-negotiable.

5.2 Evaluation Criteria

The Evaluator assesses agent behavior against the following measurable criteria:

Criterion	Measurement	Threshold Guidance
Task completion rate	Percentage of jobs completed without policy violations or TUI-escalated corrections.	Deployment-specific. Minimum recommended: 95% over evaluation window.
Operator override rate	Number of times the operator overrode, corrected, or rejected an agent output.	Lower is better. Recommended: < 5% of total actions over evaluation window.
Action scope compliance	Ratio of actions attempted within permitted scope versus out-of-scope attempts.	100% in-scope required. Any out-of-scope attempt resets the evaluation window.
Stable operation duration	Calendar time at current WAL level without demotion events.	Minimum recommended: 30 days for 0→1, 60 days for 1→2, 90 days for 2→3.
Anomaly absence	Zero anomalous behavior patterns detected.	Binary. Any confirmed anomaly resets the evaluation window.
Minimum task volume	Minimum number of evaluated tasks within the window.	Prevents gaming via inactivity. Recommended: ≥50 tasks for 0→1, ≥100 for 1→2, ≥200 for 2→3.
Risk-weighted scoring	Override and error rates weighted by task consequence class, not raw count.	Prevents dilution of high-risk failures with high-volume trivial successes.

Multi-factor validation: All criteria must be met simultaneously. No single-metric compensation. An agent cannot offset a high override rate on complex tasks with a perfect completion rate on trivial ones—risk-weighted scoring prevents this dilution.

5.3 Evaluation Output

When the Evaluator determines that all promotion criteria are met, it should emit a structured recommendation substantially equivalent to the following:

```
{
  "recommendation": "promote",
  "agent": "agent-wal1",
  "current_level": 1,
  "proposed_level": 2,
  "evaluation_window": {
    "start": "2026-02-01T00:00:00Z",
    "end": "2026-03-23T00:00:00Z"
  },
  "evidence": {
    "task_completion_rate": 0.97,
    "operator_override_rate": 0.02,
    "scope_violations": 0,
    "days_at_current_level": 51,
    "anomalies_detected": 0,
    "total_tasks_evaluated": 127,
    "risk_weighted_error_rate": 0.008
  },
  "generated_at": "2026-03-23T14:00:00Z",
  "status": "awaiting_human_authorization"
}
```

The recommendation should itself be logged in the audit trail. The operator reviews the evidence and authorizes, defers, or denies the promotion. The decision and rationale are also logged.

5.4 What the Evaluator Does Not Do

- It does not execute WAL transitions. Only the human operator does.
- It does not modify policy files or the WAL state store. It is a read-only consumer of audit logs.
- It does not run inside the agent's sandbox. It is a separate process with read access to the audit log and no write access to agent infrastructure.
- It does not automatically re-promote after demotion. Recovery follows the same criteria as initial promotion.
- It does not perform semantic analysis of traffic content. It evaluates structural compliance, not payload intent. Covert-channel defense is a separate infrastructure layer (see Section 10).

5.5 Event Taxonomy and Adjudication Semantics

v2.0 amendment. Adversarial review correctly identified that the v1.0 evaluator criteria used undefined nouns. This section defines the terms the evaluator operates on.

5.5.1 What Is a Task?

A task is a discrete unit of work submitted to the agent with a defined completion condition and a durable identifier in the audit stream. A task is "completed" when the agent produces a final output and the job transitions to a terminal state (delivered, rejected, or failed). Abandoned or indeterminate work does not count as completed.

Minimum task granularity is deployment-specific. The metric should resist gaming—implementations should avoid counting arbitrarily fragmented micro-tasks as independent evidence of reliability. Risk-weighted scoring provides the primary defense: if individual task consequence weights are near zero, the aggregate contribution to the promotion evidence is proportionally small.

5.5.2 What Is a Confirmed Material Error?

A confirmed material error is an agent output or action that: (a) produced a factually incorrect, structurally invalid, or policy-violating result, AND (b) the error was confirmed by a human operator, automated verification gate, or downstream system check—not merely flagged as suspicious. The confirming authority and confirmation method are recorded in the audit log event.

5.5.3 What Is an Operator Override?

An operator override is any event where the human operator modifies, rejects, or replaces an agent output because the output was wrong or insufficient. Routine approvals at WAL-0 and WAL-1 (where human review is expected) are not overrides. An override occurs when the operator's action indicates the agent's output was deficient. The audit log event includes a reason_code:

Reason Code	Meaning	Promotion Impact
correction	Agent output contained an error. Operator fixed it.	Counts against override rate.
rejection	Agent output was unacceptable. Operator replaced it.	Counts against override rate.
preference	Agent output was acceptable but operator preferred a different approach.	Does not count against override rate.
scope_redirect	Agent attempted an out-of-scope action. Operator redirected.	Counts as scope violation. Resets evaluation window.

5.5.4 What Is an Anomaly?

An anomaly is a deviation from the agent's established behavioral baseline that cannot be explained by the current task context. Examples: accessing filesystem paths not related to the current task, network connection attempts to endpoints not in the approved list, resource consumption spikes unrelated to workload, and timing patterns inconsistent with normal operation. Anomaly detection is structural, not semantic—it detects behavioral deviations, not intent.

False positive management: anomaly alerts should be reviewed by the operator before being classified as confirmed. Only confirmed anomalies reset the evaluation window. The confirmation decision and rationale are logged.

6. Demotion Triggers and Recovery

6.1 Demotion Policy

Demotion is not failure. It is the governance system working correctly—a control response, not a moral judgment. When an agent's behavior exceeds its trust boundaries, the system constrains it to a lower trust level and logs the event.

Demotion is automatic and immediate. It does not require human authorization. Promotion does. The asymmetry is intentional.

6.2 Trigger Classification

Trigger Category	Trigger Event	Demotion Action
Scope violation	Agent attempts an action outside its pre-approved scope not routed through TUI.	Demote one level.
Confirmed error	An action within approved scope produces a confirmed material error.	Demote one level.
Credential access	Agent attempts to access credentials, API keys, or tokens outside approved scope.	Immediate demotion to WAL-0.
Unauthorized egress	Agent attempts network communication to an unapproved endpoint.	Immediate demotion to WAL-0.
Unauthorized spawning	Agent spawns a child without permission or at a higher WAL level than itself.	Immediate demotion to WAL-0.
Resource quota breach	Agent or its children exceed the aggregate resource quota defined in policy.	Demote one level. Second occurrence: WAL-0.
Anomalous behavior	Confirmed deviation from established behavioral baseline.	Demote one level. Second occurrence: WAL-0.
Model change (major)	Major model version change, provider switch, or architecture change.	Demote to WAL-0. Restart promotion clock.
Model change (minor)	Minor model patch, quantization change, or configuration adjustment.	Demote one level. Shadow evaluation required before re-promotion.
Emergency security patch	Vendor-issued CVE fix or critical security update.	Hold current WAL level. Enter shadow evaluation mode (Section 6.5).

6.3 Recovery Path

Recovery from demotion follows the same path as initial promotion. There is no fast track:

- The evaluation window resets at the time of demotion.
- The agent must meet all promotion criteria at the demoted level before re-promotion eligibility.
- Re-promotion requires human authorization, same as initial promotion.

- The demotion event, recovery period, and re-promotion decision are all logged with full context.

There is no automatic re-promotion. An agent demoted from WAL-2 to WAL-1 must meet all WAL-1 to WAL-2 promotion criteria again, with a fresh evaluation window, and receive human authorization before returning to WAL-2.

6.4 Model Change Taxonomy

v2.0 amendment. The v1.0 specification applied blanket WAL-0 demotion to all model changes. Adversarial review correctly identified this as operationally unsustainable: organizations that patch models for security vulnerabilities should not face a choice between security updates and workflow continuity.

The v2.0 model change taxonomy distinguishes three categories:

- **Major changes** (new model version, different provider, architecture change, training data update): Demote to WAL-0. The behavioral impact is unpredictable. The new model must demonstrate reliability from scratch. The trust earned by the previous version does not transfer.
- **Minor changes** (minor version patch, quantization adjustment, inference configuration change): Demote one level. Shadow evaluation (Section 6.5) is required before re-promotion. The behavioral impact is expected to be small but must be verified.
- **Emergency security patches** (vendor-issued CVE fixes, critical security updates): Hold current WAL level. Enter shadow evaluation mode. If shadow evaluation detects behavioral deviation, demote one level. If shadow evaluation confirms behavioral consistency, maintain current level. This prevents organizations from choosing between security and operational continuity.

The change category is determined by the operator based on the vendor's change documentation. If the category is ambiguous, default to major.

6.5 Shadow Evaluation Mode

v2.0 addition. Shadow evaluation is a time-bounded verification period following a minor model change or emergency patch. During shadow evaluation:

- The agent continues operating at its current WAL level.
- All outputs are additionally logged to a shadow evaluation stream for comparison against pre-change baselines.
- The Escalation Evaluator runs a behavioral consistency check: task completion rate, override rate, and action patterns are compared against the agent's pre-change performance at the same WAL level.
- If behavioral deviation exceeds a configurable threshold, automatic demotion triggers.
- If the shadow evaluation window completes without deviation, the agent's WAL level is confirmed and normal operation resumes.
- Default shadow evaluation window: 7 days for minor changes, 3 days for emergency security patches.

7. Multi-Agent WAL Independence

Each agent's WAL position is independent. The same OpenShell infrastructure can support agents at different WAL levels simultaneously, each governed by its own policy file, WAL state store, and audit stream.

7.1 Practical Implications

- A well-established workflow agent at WAL-2 can run alongside a newly deployed agent at WAL-0 on the same host, each with its own sandbox and policy.
- Promoting one agent does not affect the trust level of any other agent.
- Demoting one agent does not affect any other agent unless a parent-child relationship exists (child agents are automatically demoted if their parent is demoted).
- Organizations choose their own escalation velocity. Aggressive organizations promote agents within days. Government environments may take months. Both use identical infrastructure.

7.2 Per-Capability WAL Granularity

Within a single agent, WAL levels can be assigned per capability rather than per agent. A scheduling capability can earn WAL-2 while a financial operations capability remains at WAL-0. This limits the blast radius of any single capability's promotion.

7.3 Per-Capability Isolation Requirements

v2.0 clarification. Adversarial review correctly identified that scoped action classes within a single sandbox do not provide true isolation if the agent shares memory, scratch directories, or credentials across capabilities. The following isolation requirements apply:

- **Low-consequence capability families** (all capabilities have similar risk profiles): Scoped action classes within a single sandbox are sufficient. Policy-level separation of permitted actions provides adequate boundary.
- **Mixed-consequence domains** (capabilities span significantly different risk levels): Per-capability WAL requires separate sandboxes, separate credential namespaces, and separate state partitions. A WAL-2 scheduling capability must not share credentials, memory, or scratch space with a WAL-0 financial operations capability. If true isolation is not achievable, the entire agent operates at the lowest WAL level of any of its capabilities.

The operator determines whether a deployment is low-consequence or mixed-consequence. When in doubt, default to separate sandboxes.

8. Dual-Route Deployment Model

The WAL integration profile supports two deployment postures on identical infrastructure:

8.1 Velocity-First Deployment

Organizations that want immediate autonomous agents start at WAL-2 or WAL-3. This is the current OpenShell default posture. The operator writes a permissive policy, deploys the agent, and manages exceptions through the TUI. The WAL framework is present but the starting position is near the top.

This is appropriate for development environments, internal tooling, and organizations with high risk tolerance and strong operational monitoring.

8.2 Trust-First Deployment

Organizations that need graduated trust start at WAL-0 and progress upward. Every capability begins under full human oversight. Promotion is earned through measured reliability. Demotion is automatic on violation. The audit trail documents every transition.

This is required for regulated industries (government, healthcare, finance, defense), organizations undergoing FedRAMP or equivalent accreditation, and any deployment where the consequences of autonomous agent failure include regulatory action, patient harm, or national security implications.

8.3 Same Infrastructure, Different Starting Points

Both deployment models use identical OpenShell infrastructure. The difference is the starting WAL state and the escalation criteria:

- No separate “enterprise edition” is required for regulated environments.
- Organizations can run velocity-first and trust-first agents on the same infrastructure simultaneously.
- A startup that begins with velocity-first deployment can transition to trust-first as it enters regulated markets—without changing its agent infrastructure.

This dual-route capability is what makes OpenShell viable for both Silicon Valley velocity and federal accreditation timelines.

9. Implementation Path

All phases are additive. They do not require changes to OpenShell's core architecture. Each phase should deliver standalone value.

9.1 Phase 1: WAL Policy Preset Collection and State Store

Deliverable: Four policy profiles (wal-0-recommend.yaml through wal-3-autonomous.yaml), a WAL state-store schema or template, and startup reconciliation logic, published as an OpenShell preset collection in nemoclaw-blueprint/policies/presets/wal/.

Scope: Policy files, state store schema, and reconciliation logic. No code changes to OpenShell or NemoClaw core. Documentation includes a WAL Quick Start guide and the restart reconciliation protocol.

Effort: Low-to-medium. YAML authoring, JSON schema definition, a shell script for startup reconciliation, and documentation.

9.2 Phase 2: Escalation Evaluator Module

Deliverable: A standalone module (Python or TypeScript) that reads OpenShell audit logs, evaluates agent behavior against configurable promotion criteria with the event taxonomy defined in Section 5.5, and outputs structured promotion recommendations.

Scope: Read-only log consumer. Implements risk-weighted scoring, minimum task volume requirements, and behavioral consistency checks for shadow evaluation. Outputs JSON recommendations for human review.

Effort: Medium. Requires parsing OpenShell's audit log format, implementing the evaluation criteria and event taxonomy, and building the shadow evaluation comparison engine.

9.3 Phase 3: NemoClaw CLI Integration

Deliverable: Native WAL commands in the NemoClaw CLI for managing agent trust levels.

```
nemoclaw <agent> wal status          # Current WAL level, time at level, last
transition
nemoclaw <agent> wal promote         # Review evaluator recommendation, authorize
promotion
nemoclaw <agent> wal demote [level]  # Manual demotion with reason logging
nemoclaw <agent> wal history         # Full WAL transition history from state store
nemoclaw <agent> wal evaluate        # Run the escalation evaluator on demand
nemoclaw <agent> wal shadow-status   # Shadow evaluation progress and deviation
metrics
```

Scope: CLI wrapper around the policy swap mechanism, WAL state store, and Escalation Evaluator. The promote command should require interactive operator confirmation—it should not be scriptable without an explicit --yes flag, and even then, the authorization is logged.

Effort: Medium. Requires NemoClaw plugin development and integration with the Escalation Evaluator.

9.4 Phase Dependency

Each phase builds on the previous but delivers standalone value:

- **Phase 1 alone** gives operators structured policy profiles, durable trust-state semantics, and restart reconciliation. Useful immediately, even without an evaluator.
- **Phase 1 + Phase 2** adds automated evaluation of agent readiness for promotion, risk-weighted scoring, and post-change shadow evaluation.
- **Phase 1 + Phase 2 + Phase 3** provides a fully integrated WAL management experience within the NemoClaw CLI. The governance model should remain valid even if the CLI wrapper is never built.

10. Acknowledged Limitations and Deferred Concerns

The following concerns were raised in adversarial review. They are valid but out of scope for this specification. They are documented here so the proposal is not read as silently assuming them away.

10.1 Demotion Race Condition

Concern: When a demotion event triggers a policy swap, OpenShell's handling of in-flight operations (active subprocesses, open network sockets, partially written files) during the swap is undefined in the current alpha documentation.

Position: This specification defines the required control outcome (Section 4.5.3). The runtime or an implementation wrapper must enforce it. Until that is verified, in-flight transition handling remains an implementation risk rather than a closed issue.

10.2 Covert Channel Defense

Concern: An agent could exfiltrate data by encoding it in DNS lookups, HTTP headers, or authorized API call parameters. The audit log would show 100% policy compliance while the agent is actively hostile.

Position: Covert-channel defense is a defense-in-depth concern that applies to any sandboxed agent, not specifically to WAL. WAL governs trust progression based on structural compliance. Semantic traffic analysis, DNS inspection, and payload anomaly detection are separate infrastructure layers. WAL's contribution is ensuring that an agent starts under full oversight (WAL-0) where covert channels are less consequential because all actions are human-reviewed.

10.3 State Rollback and Downstream Impact

Concern: When an agent is demoted due to a confirmed error, the demotion constrains future actions but does not fix damage already done. Downstream systems that consumed erroneous output are not automatically notified.

Position: WAL is a trust governance framework, not an incident response system. Demotion is a containment action—it prevents further autonomous damage. Downstream impact assessment, notification, and remediation are operational responsibilities that belong to the organization's incident response process, not to the trust state machine. The audit log provides the forensic foundation for that process.

10.4 Alert Fatigue at WAL-0

Concern: In high-volume environments, WAL-0 and WAL-1 generate many TUI prompts, risking operator alert fatigue and rubber-stamp approvals.

Position: WAL-0 is explicitly an evaluation period, not a sustained production posture. The design intent is to demonstrate competence and graduate upward. Organizations deploying agents in high-volume environments should set aggressive but evidence-based promotion timelines. WAL-1 substantially reduces alert volume by allowing autonomous read operations and only surfacing write operations. The progression from WAL-0 to WAL-1 is the primary alert-fatigue mitigation.

10.5 Context Packaging and Data Autonomy

Concern: WAL levels govern action autonomy (what the agent can do) but do not fully specify data autonomy (what context the agent includes in cloud inference requests). OpenShell's Privacy Router governs where inference goes, but not what payload is safe to include.

Position: Valid. A companion context-packaging or memory-selection policy is needed for deployments where contextual sensitivity is the primary risk (healthcare, legal, government). The DRNT architecture's Context Packager provides a reference model: a structural gate that strips sensitive data before cloud dispatch, with per-task-type payload templates and default-deny for unclassified content. Integrating a Context Packager profile into the WAL policy files is a natural future extension.

10.6 Audit Log Integrity

Concern: If an agent achieves a sandbox escape, it could alter the audit logs or WAL state store to erase evidence of anomalies.

Position: The DRNT audit event schema (Spec 1) specifies SHA-256 hash chaining, append-only storage with chattr +a, a separate log writer process, and read-only mounts for all containers except the log writer. These controls should be applied to the OpenShell audit log and the WAL state store. The WAL state store should be mounted read-only to the sandbox and writable only by the WAL governance process. If OpenShell's current alpha implementation does not enforce append-only audit storage, this is an implementation gap that must be closed before WAL-2 or WAL-3 operation in regulated environments.

11. Conclusion

OpenShell provides the most promising publicly available enforcement foundation for autonomous agents. Its Sandbox, Policy Engine, and Privacy Router deliver genuine out-of-process containment. Its declarative policy model, hot-reloadable at runtime, gives operators fine-grained control over agent behavior. Its audit trail captures every enforcement decision. These are the right primitives.

What OpenShell does not yet provide is the governance semantics that organize those primitives into a systematic, auditable trust lifecycle. The current model—start with a policy and adjust manually—works for development and experimentation. It does not satisfy the accountability requirements of the regulated environments that represent the largest and most consequential market for autonomous agent infrastructure.

The WAL framework provides the missing governance layer: graduated trust with measurable promotion criteria, automatic demotion on violation, per-agent and per-capability independence, durable state that survives runtime restarts, a model change taxonomy that balances security updates against operational continuity, and a human-authorized transition model with defined event semantics.

The integration is clean. Each WAL level is a policy file. Each transition is a policy swap governed by a durable state store. The Escalation Evaluator reads audit logs and surfaces recommendations with risk-weighted evidence. The human decides. No changes to OpenShell's core architecture are required.

The cage is built. This specification provides the ladder.

End of Specification